Notes de synthèse – TP2 : PPC avec MiniZinc

1. Les n reines

Comme `q[i]` donne la ligne de la reine sur la colonne i, la contrainte de colonne est prise en compte par la structure des variables. On ajoute `alldifferent` : sur `q[i]` (lignes), `q[i] + i` (diagonales montantes) et `q[i] - i` (descendantes). Nombres de solution : 2 pour $n = 4$ et 92 pour $n = 8$.

2. Sudoku et contraintes redondantes

Le modèle applique `alldifferent` sur les lignes, les colonnes et les blocs 3×3. La grille à 30 indices du cours donne une solution unique.

Question bonus ($\Sigma = 45$) : Forcer la somme de chaque ligne, colonne et bloc à 45 ne change pas la solution parce que tous les nombres dont differents (entre 1 - 9) (9 cases au total).

Pourtant, cette répetition rend le solveur plus rapide.

Parallèle avec le **feature engineering en Machine Learning** : dans une régression polynomiale, quand on crée la variable $x_1 \cdot x_2$ ou x^2, on redonne cette information au modèle sous une forme différente il peut apprendre plus sur le jeu de données.

3. Ordonnancement (ExedBike)

On trouve un makespan optimal de 12h. Il y a qu'une seule machine, les 3 lots s'enchaînent sans pause ($3 + 5 + 4 = 12$h).

MiniZinc vs OR-Tools : MiniZinc est plus rapide faire des tests, OR-Tools pour la compatibilité avec python en production.